A_

// AUDITNET · AUDITORIA DE CONFIGURAÇÕES DE REDE

Manual de instalação e desenvolvimento

Arquitetura, instalação, testes, implantação no Azure e como estender a plataforma.

Versão 1.0 · outubro 2026
Português

Sumário

Introdução

Arquitetura

- Componentes
- Decisões-chave

Repositório e convenções

- Camadas de cada serviço
- Convenções

Instalação local

- Requisitos
- 1. Variáveis de ambiente
- 2. Bancos e serviços de apoio
- 3. Serviços
- 4. Console
- 5. Dados de demonstração (opcional)

Testes

- Testes dos serviços
- Testes de tela
- No CI

Como estender a plataforma

- Um endpoint novo
- Uma migração
- Um tipo de regra de auditoria
- Uma tela
- Uma tradução
- Um e-mail
- Os manuais

Integração contínua e implantação no Azure

- GitHub Actions
- Recursos no Azure
- Criar tudo do zero
- Implantar uma versão manualmente
- Segurança da implantação

Operação

- Modo demo e custos

Backups
Revisão periódica
E-mail
Diagnóstico
Dados de demonstração no Azure
Problemas reais e como foram resolvidos
Permissões por perfil e onde são controladas
Onde é controlado

Introdução

O AuditNet é uma plataforma web B2B para auditar configurações de equipamentos de rede Cisco IOS/IOS-XE. Ela compara a configuração real de cada equipamento (running-config) com uma configuração esperada (baseline), detecta os desvios, prioriza-os por severidade e pela criticidade do equipamento, gera alertas e relatórios e preserva a evidência de cada auditoria. É multiorganização: cada cliente vê apenas os seus dados.

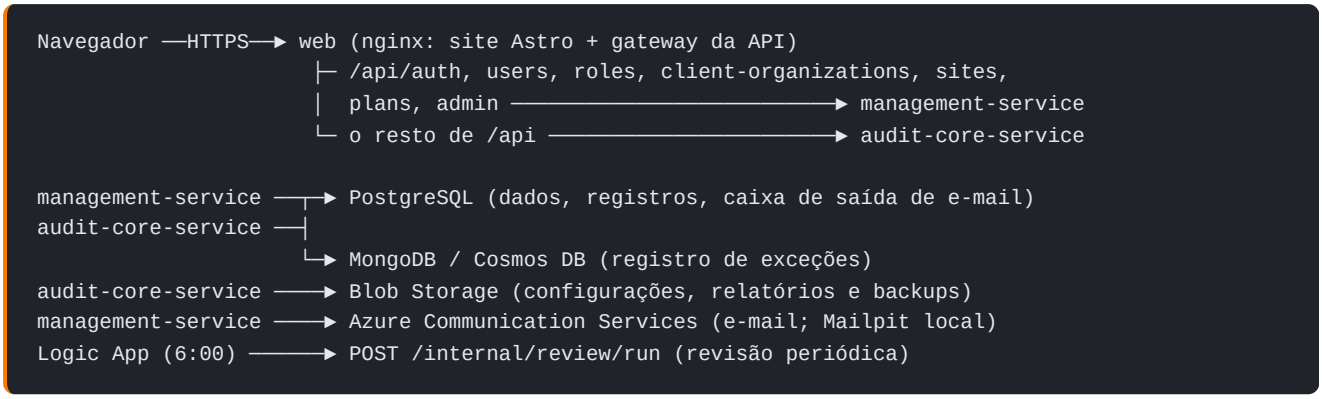
Este manual é para quem precisa **instalar, operar ou modificar** a plataforma: desenvolvedores que entram no projeto e quem a implanta no Azure. Ele cobre:

- a arquitetura e por que ela é montada assim;
- como o repositório está organizado e quais convenções segue;
- como executá-la em um computador local e carregar dados de demonstração;
- como rodar os testes;
- como adicionar um endpoint, uma migração, uma tela, uma tradução ou um e-mail;
- o fluxo de integração e implantação no Azure e a operação diária;
- os problemas reais que surgiram durante o desenvolvimento e como foram resolvidos;
- o que cada perfil pode fazer e onde isso é controlado no código.

DADO	VALOR
Repositório	`github.com/Leonel-Caruso/tfi-auditnet`
Branches	`main` (versão vigente, implantada no Azure) e `test` (testes, não é implantada); `entrega-2` fica como histórico
Decisões de projeto	`docs/entrega-2/REGISTRO-DECISIONES.md` (D-01 em diante)
Migrações	`docs/database/MIGRACIONES.md`
Implantação	`docs/deploy/AZURE.md`

> Nos comandos, o que está entre `<` e `>` é substituído pelo valor real (sem os sinais). Senhas e segredos nunca são escritos no repositório nem na linha de comando: os scripts os pedem ao serem executados.

Arquitetura



› Componentes

COMPONENTE	TECNOLOGIA	RESPONSABILIDADE
web	nginx + Astro 5 (site estático)	Serve o console e encaminha `/api/...` para cada serviço na mesma origem: os serviços não são expostos e não é preciso CORS. Publica a política de segurança de conteúdo.
management-service (porta 8081)	Java 21, Quarkus 3	Acesso e JWT, usuários e perfis, organizações e sedes, planos, e-mail, registro do sistema.
audit-core-service (porta 8082)	Java 21, Quarkus 3	Dispositivos, configurações, baselines e regras, motor de auditoria, achados, alertas, painel, relatórios, revisão periódica, arquivos e backups.
PostgreSQL 17	Flyway por serviço	Um banco compartilhado; cada serviço migra as suas tabelas com a sua própria tabela de histórico.
MongoDB / Cosmos DB	API do MongoDB 4.2	Registro de exceções (a entidade NoSQL).
Blob Storage / Azurite	contêiner privado	Arquivos com o seu SHA-256.

› Decisões-chave

- **Dois serviços e um banco.** Eles são separados por responsabilidade (acesso e administração de um lado, auditoria do outro) e compartilham o PostgreSQL para que as consultas de escopo (organização, plano) sejam simples. O audit-core lê o plano com uma consulta nativa; por isso o management migra primeiro.
- **Segurança.** O login devolve um JWT assinado (HS256) com o usuário, a sua organização e os seus perfis. Cada endpoint declara os perfis permitidos com `@RolesAllowed` e cada serviço aplica o escopo por organização. Mudar o perfil, a organização ou a senha invalida as sessões abertas.
- **Três registros.** Sistema (acessos e segurança) e transações (valor anterior e novo de cada operação de negócio) no PostgreSQL; exceções no MongoDB com nível configurável. Um código de rastreio (`X-Request-ID` , colocado pelo nginx) relaciona uma mesma operação nos três.

- **Evidência inalterável.** Configurações, auditorias, acompanhamentos de achados e relatórios não são modificados: há triggers somente de inserção e os relatórios são um retrato fixo.
- **E-mail com caixa de saída.** Os avisos são salvos em `correos_salientes` dentro da mesma transação do fato que os origina e são despachados depois, com novas tentativas. Assim, uma falha do provedor não perde avisos.
- **Multi-idioma.** O console traduz no navegador com dicionários; as mensagens de erro são traduzidas com um catálogo conforme o `Accept-Language`; os e-mails usam o idioma do destinatário.

Repositório e convenções

```
backend/management-service/   Quarkus: acesso, usuários, organizações, planos, e-mail
backend/audit-core-service/   Quarkus: inventário, baselines, auditoria, achados, relatórios
frontend/web-app/             Astro: páginas, scripts, dicionários, estilos e nginx
docker/                       PostgreSQL, MongoDB, Azurite e Mailpit para desenvolvimento local
infra/azure/                  Scripts de criação e operação no Azure (01 a 09)
scripts/                      load-env.ps1 e datos-demo/ (carga de demonstração)
tests/pantallas/              Testes de tela (Playwright e API simulada)
tests/datos_demo/             Testes dos dados de demonstração
tests/carga/                  Teste de carga com k6
docs/                         Decisões, migrações, implantação e manuais
.github/workflows/            CI, imagens e implantação
```

› Camadas de cada serviço

```
ar/edu/uai/tfi/<servico>/
  api/rest/         recursos JAX-RS e DTO: validam a entrada, @RolesAllowed e escopo
  application/      casos de uso e portas (application/port), sem infraestrutura
  domain/           modelo (records e enums) e repositórios como interfaces
  infrastructure/   adaptadores: Panache, segurança, e-mail, registros, arquivos, i18n
```

As dependências apontam para dentro: `api → application → domain`, e `infrastructure` implementa as portas do domínio e da aplicação. Por isso os serviços de aplicação são testados com implementações em memória (`src/test/.../support`).

› Convenções

- **Nomes em espanhol** no domínio, nos serviços e nos textos; as rotas da API em inglês (`/api/devices`, `/api/findings`).
- **Código simples e defensável:** sem comentários extensos; só os que explicam um porquê.
- **Migrações Flyway** em `src/main/resources/db/migration` de cada serviço: `V<n>__<descricao>.sql`. Uma migração já aplicada nunca é modificada; as mudanças vão em uma nova. Histórico: `flyway_historial_management` e `flyway_historial_audit_core`.
- **Decisões** em `REGISTRO-DECISIONES.md` com número `D-xx`: o que foi decidido, por quê, alternativas e teste.
- **Textos da interface** nunca escritos no HTML nem nos scripts: sempre a partir dos dicionários (veja *Como estender a plataforma*).
- **Sem segredos no Git:** `.env` é ignorado; `.env.example` lista as variáveis sem valores.

Instalação local

› Requisitos

FERRAMENTA	VERSÃO
Docker Desktop	com Docker Compose
Java (JDK)	21
Maven	3.9 ou superior
Node.js	24
Python	3.11 ou superior (testes de tela, dados de demonstração e manuais)
PowerShell	5 ou superior (Windows); no Linux ou macOS, os mesmos passos com `bash`

› 1. Variáveis de ambiente

Copie `.env.example` para `.env` e preencha os valores locais: banco de dados, `JWT_SECRET`, o administrador inicial (`BOOTSTRAP_ADMIN_*`) e a organização inicial (`BOOTSTRAP_ORG_*`). O `.env` não vai para o Git.

› 2. Bancos e serviços de apoio

```
.\scripts\load-env.ps1
docker compose --env-file .env -f docker/compose.database.yaml up -d
docker ps
```

Sobe `tfi-postgres` (5432), `tfi-mongo` (27017), `tfi-azurite` (10000, armazenamento) e `tfi-mailpit` (os e-mails são lidos em `http://localhost:8025`).

› 3. Serviços

Em um terminal, **primeiro o management-service** (aplica as suas migrações e cria os perfis, a organização e o administrador iniciais):

```
.\scripts\load-env.ps1
cd .\backend\management-service
mvn quarkus:dev
```

Quando aparecer `Listening on: http://localhost:8081`, em outro terminal:

```
.\scripts\load-env.ps1
cd .\backend\audit-core-service
mvn quarkus:dev
```

› 4. Console

```
cd .\frontend\web-app
npm install
npm run dev -- --host localhost
```

Abra `http://localhost:4321` e entre com o administrador do `.env`. Em desenvolvimento, o navegador chama os serviços nas portas 8081 e 8082 (não passa pelo nginx).

› 5. Dados de demonstração (opcional)

`scripts/datos-demo` carrega 6 organizações fictícias com sedes, usuários, equipamentos, baselines, regras e cerca de 100 auditorias, usando a API como faria um administrador:

```
python scripts\datos-demo\cargar.py --local --admin <usuario_admin> --correo
<caixa>@gmail.com
docker cp .\scripts\datos-demo\historia.sql tfi-postgres:/tmp/historia.sql
docker exec tfi-postgres sh -c 'psql -U $POSTGRES_USER -d $POSTGRES_DB -f
/tmp/historia.sql'
python scripts\datos-demo\cargar.py --local --admin <usuario_admin> --correo
<caixa>@gmail.com --reportes
```

- `cargar.py` pede a senha do administrador e a dos novos usuários; não as salva. Os e-mails dos usuários são aliases da caixa (`caixa+usuario@dominio`) e os seus avisos de auditoria ficam desligados.
- `historia.sql` distribui nas últimas 8 semanas as auditorias recém-carregadas com os seus achados e alertas.
- `--reportes` gera os relatórios depois de distribuídas as datas.
- Pode ser executado de novo: o que já existe é reutilizado.

Testes

O QUÊ	COMANDO	ONDE
Unitários e de integração de cada serviço	`mvn test`	em `backend/<servico>`
Tipos e dicionários do frontend	`npm run check`	em `frontend/web-app`
Testes de tela	`npm run build:pruebas` e `python -m pytest tests\pantallas`	build em `frontend/web-app`, pytest a partir da raiz
Dados de demonstração	`python -m pytest tests\datos_demo`	raiz
Carga (k6)	veja `docs/entrega-2/CARGA.md`	raiz

› Testes dos serviços

Os serviços de aplicação são testados sem banco de dados, com repositórios em memória (`support/EnMemoria...`) e um relógio fixo. Cada decisão do registro indica qual teste a respalda. Antes de enviar mudanças, os dois serviços devem passar no `mvn test` completo.

› Testes de tela

`tests/pantallas` abre o site compilado no Chromium (Playwright) contra uma API simulada (`servidor.py`, dados em `datos.py`) e percorre todas as telas com os cinco perfis: que carreguem sem erros, que não mostrem textos técnicos, que caibam em um celular de 390 px, que estejam traduzidas para inglês e português e que o tema escuro não deixe fundos claros.

```
cd .\frontend\web-app
npm run build:pruebas
cd ..\..
python -m pip install -r tests\pantallas\requirements.txt
python -m playwright install chromium
python -m pytest tests\pantallas
```

- `build:pruebas` compila o site para chamar a API na mesma origem, como atrás do nginx.
- O servidor de testes envia a mesma política de segurança de conteúdo que o nginx: um script escrito dentro do HTML faz o teste falhar. Por isso, para esperar uma condição usa-se `pagina.esperar("<expressao>")` e não `wait_for_function`.
- Para uma tela nova: adicionar a rota em `test_pantallas.py` e, se ela chamar endpoints novos, a sua resposta em `RUTAS` de `servidor.py`.

› No CI

Cada push para `main` ou `test` executa, no GitHub Actions, os testes dos dois serviços, a verificação e o build do frontend, os testes de tela e os dos dados de demonstração. Se algo falhar, as imagens não

são publicadas.

Como estender a plataforma

› Um endpoint novo

1. **Domínio:** se necessário, um record em `domain/model` e os métodos na interface de `domain/repository`.
2. **Aplicação:** o caso de uso em um serviço de `application/<modulo>`. Ele valida, aplica o escopo da organização e registra a operação no registro de transações (`TrazabilidadPort`, com o valor anterior e o novo).
3. **Infraestrutura:** a implementação do repositório com Panache em `infrastructure/persistence`.
4. **API:** o recurso em `api/rest` com `@Path`, os DTO como records em `api/rest/dto` e `@RolesAllowed` com os perfis que podem usá-lo.
5. **Permissões:** se for uma ação nova, adicioná-la a `MatrizPermisos` (management) e a `RolServiceTest`; a matriz é mostrada na tela de Usuários.
6. **Gateway:** se o prefixo da rota for novo e pertencer ao management-service, adicioná-lo à expressão de `location ~ ^/api/(auth|users|...)` em `frontend/web-app/nginx/default.conf.template`. O resto vai para o audit-core.
7. **Mensagens:** cada nova mensagem de erro vista pelo usuário vai em `src/main/resources/i18n/mensajes.txt` do serviço, com as linhas `es:`, `en:` e `pt:` (se faltar, é mostrada em espanhol).
8. **Testes:** do serviço com os repositórios em memória e, se uma tela a usar, a resposta simulada em `tests/pantallas/servidor.py`.

› Uma migração

1. Criar `V<n>__<descricao>.sql` em `db/migration` do serviço dono da tabela, com o número seguinte ao último **desse** serviço.
2. Ajustar a entidade Panache e o repositório.
3. Adicionar a linha em `docs/database/MIGRACIONES.md`.
4. Ao iniciar (`mvn quarkus:dev`, ou a implantação no Azure), o Flyway a aplica sozinho. Nunca editar uma já aplicada.

› Um tipo de regra de auditoria

Os tipos são `DEBE_CONTENER`, `NO_DEBE_CONTENER` e `VALOR_ESPERADO`. Para adicionar um:

1. O valor em `TipoReglaConfiguracion` e a sua avaliação em `MotorAuditoriaService` (devolve se cumpre e a evidência).
2. Uma migração que amplie o `CHECK` de `reglas_baseline.tipo`.
3. No console, a opção no formulário de Regras, o seu nome no dicionário (`condicion.<TIPO>`) e, se a evidência tiver uma frase nova, a sua tradução em `lib/evidencia.ts`.
4. Testes do motor com configurações que cumprem e que não cumprem.

› Uma tela

1. `src/pages/<rota>.astro` com `MasterLayout` (o atributo `i18n` indica o prefixo das chaves do cabeçalho) e o seu script em `src/scripts/<rota>.ts`, carregado com `<script src="../scripts/<rota>.ts">`. Não se escrevem scripts dentro do HTML: a política de segurança não os permite.
2. O item do menu em `components/Sidebar.astro` (código curto e perfis) e a rota na lista de `components/Topbar.astro`; opcionalmente, palavras para *Ir para módulo* em `layouts/MasterLayout.astro`.
3. As chaves `menu.<rota>` e `ruta.<rota>` em `i18n/es.ts`, `en.ts` e `pt.ts`, e um dicionário `i18n/pantallas/<tela>.ts` com `es`, `en` e `pt`, adicionado em `i18n/index.ts`.
4. A rota em `tests/pantallas/test_pantallas.py` (`PANTALLAS` ou `PANTALLAS_POR_PERFIL`).

› Uma tradução

- No HTML: `data-i18n="chave"` para o texto e `data-i18n-placeholder`, `data-i18n-title`, `data-i18n-aria-label` ou `data-i18n-label` para atributos.
- Nos scripts: `t('chave', { dado })`, `plural(n, 'chave')` (chaves `chave.uno` e `chave.otros`) e `etiqueta('grupo', codigo)` para códigos como severidades ou estados.
- O tipo `Textos<typeof es>` obriga inglês e português a terem as mesmas chaves: `npm run check` falha se faltar uma. Os dados carregados pelos usuários (nomes de regras, sedes) não são traduzidos.

› Um e-mail

Os textos estão em `management-service/src/main/resources/correo/textos_<idioma>.properties` e a montagem em `PlantillasCorreo`. Um tipo novo precisa do seu valor em `TipoCorreo`, dos seus textos nos três idiomas e de um teste em `PlantillasCorreoTest`. Ele é enfileirado com `CorreoService.encolar(...)` (ou, a partir do audit-core, na caixa de saída comum) e sai com as novas tentativas de sempre.

› Os manuais

As fontes estão em `docs/manuales/<idioma>/usuario` e `docs/manuales/<idioma>/desarrollo`, em Markdown. As capturas de tela são feitas automaticamente a partir do site de testes com dados fictícios (`capturas.py`, `datos_manual.py`):

```
cd .\frontend\web-app; npm run build:pruebas; cd ..\..
python -m pip install -r docs\manuales\requirements.txt
python docs\manuales\generar.py
```

Os manuais do usuário ficam em `frontend/web-app/public/manuales` (o console os oferece para download) e este em `docs/manuales/pdf`.

Integração contínua e implantação no Azure

› GitHub Actions

`.github/workflows/imagenes.yml` roda em cada push para `main` ou `test` que altere código, testes ou o workflow:

JOB	O QUE FAZ
Tests	`mvn test` dos dois serviços
Frontend	`npm ci`, `npm run check` e build
Testes de tela	Playwright contra o site compilado, e os dos dados de demonstração
Imagens	Publica as 3 imagens no GitHub Container Registry com as tags `sha-<commit>` e a branch
Implantar no Azure	Só a partir de `main`: espera a **aprovação** do ambiente `azure`, entra no Azure por OIDC (sem senhas), atualiza as 3 Container Apps para `sha-<commit>` e faz um teste de fumaça

Para aprovar: *Actions → a execução → Review deployments → azure → Approve and deploy*.

› Recursos no Azure

RECURSO	NOME	PARA QUÊ
Grupo de recursos	`rg-auditnet` (Brazil South)	Tudo da plataforma
PostgreSQL Flexible Server	`psql-auditnet-lcaruso`, banco `tfi_auditnet`	Dados e registros (B1ms, cópias de 7 dias)
Container Apps	`ca-auditnet-web`, `ca-auditnet-management`, `ca-auditnet-core` em `cae-auditnet`	O console com entrada pública e os dois serviços com entrada interna
Cosmos DB for MongoDB	`cosmos-auditnet-lcaruso`	Registro de exceções (plano gratuito)
Storage	`stauditnetlcaruso`, contêiner `auditnet-archivos`	Repositório de arquivos
Communication Services	domínio gerenciado pelo Azure	E-mail (5 por minuto e 10 por hora)
Logic App	revisão diária às 6:00	Revisão periódica

› Criar tudo do zero

No **Azure Cloud Shell (Bash)**, com o repositório clonado:

```
git clone -b main https://github.com/Leonel-Caruso/tfi-auditnet.git
cd tfi-auditnet/infra/azure
bash 01-crear-base.sh            # grupo, PostgreSQL, banco e ambiente de Container Apps
bash 02-crear-apps.sh            # as 3 Container Apps a partir das imagens publicadas
bash 04-crear-cosmos.sh          # registro de exceções
bash 05-crear-storage.sh         # repositório de arquivos
bash 06-configurar-github-oidc.sh  # implantação a partir do GitHub sem senhas
bash 08-crear-email.sh           # e-mail
bash 09-crear-revision.sh        # revisão periódica
```

Os scripts pedem as senhas ao serem executados e as salvam como *secrets* das Container Apps; não há segredos no repositório. Podem ser executados de novo. `variables.sh` tem os nomes dos recursos e registra os provedores necessários.

› Implantar uma versão manualmente

```
cd ~/tfi-auditnet/infra/azure && bash 03-actualizar-imagenes.sh sha-<commit>
```

Cada serviço aplica as suas migrações ao iniciar (o management primeiro). O painel de administração mostra a tag implantada.

› Segurança da implantação

- HTTPS obrigatório com HSTS; o nginx adiciona `Content-Security-Policy`, `Permissions-Policy`, `X-Frame-Options` e `Referrer-Policy`.
- Os serviços não têm entrada pública: só se chega a eles pelo nginx. `/internal` não é publicado, exceto a revisão periódica, que exige um segredo de 256 bits.
- O PostgreSQL aceita conexões de serviços do Azure (incluindo o Cloud Shell). Para se conectar a partir de um PC, é preciso uma regra de firewall temporária com o seu IP.
- O GitHub Actions só pode atualizar as 3 apps, e cada implantação exige aprovação manual.

Operação

› Modo demo e custos

As Container Apps escalam para zero quando não há tráfego: a primeira requisição leva alguns segundos para acordá-las. O PostgreSQL pode ficar parado para economizar crédito. Antes de uma apresentação:

```
cd ~/tfi-auditnet/infra/azure
bash 07-modo-demo.sh on       # 1 réplica mínima e PostgreSQL ligado
bash 07-modo-demo.sh estado
bash 07-modo-demo.sh off      # ao terminar: volta a escalar para zero
```

O modo `on` consome a cota mesmo sem tráfego. Os custos são acompanhados no *Cost Management* do portal.

› Backups

No **Painel de administração → Gerar backup**: um ZIP com os dados de todas as tabelas, a versão do esquema e o script de restauração, salvo no repositório de arquivos. O Azure também mantém cópias automáticas do PostgreSQL por 7 dias. Como restaurar: `docs/database/RESPALDOS.md`.

› Revisão periódica

Uma Logic App chama `POST /internal/review/run` todos os dias às 6:00 (horário da Argentina). Para cada organização ativa: reaudita os equipamentos cuja última auditoria ultrapassou o prazo, avisa os achados vencidos e os equipamentos sem configuração nova, e a primeira revisão de cada mês envia o resumo do mês anterior. Pode ser disparada manualmente no painel de administração. Se o PostgreSQL estava parado, a execução desse dia falha e a seguinte coloca tudo em dia.

› E-mail

Os avisos passam pela caixa de saída (`correos_salientes`); o painel de administração mostra os pendentes, enviados e com falha, e **Enviar pendentes** força o envio. O domínio gerenciado do Azure permite poucos envios por hora: com muitos cadastros seguidos, os e-mails saem aos poucos. Localmente, são lidos no Mailpit (`http://localhost:8025`).

› Diagnóstico

1. **Painel de administração:** estado de cada serviço e dos seus componentes.
2. **Registros:** buscar o código de rastreio da operação e filtrar os três registros com ele.
3. **Logs das apps:** `az containerapp logs show -g rg-auditnet -n ca-auditnet-management --tail 100` (ou `ca-auditnet-core`, `ca-auditnet-web`).

› Dados de demonstração no Azure

No Cloud Shell (o diretório não é mantido entre sessões: clonar ou atualizar o repositório a cada vez):

```
cd ~ && git clone -b main https://github.com/Leonel-Caruso/tfi-auditnet.git; cd ~/tfi-
auditnet && git pull
SITIO=https://$(az containerapp show -g rg-auditnet -n ca-auditnet-web --query
properties.configuration.ingress.fqdn -o tsv)
python3 scripts/datos-demo/cargar.py --url $SITIO --admin <usuario_admin> --correo
<caixa>@gmail.com
export PGPASSWORD=$(az containerapp secret show -g rg-auditnet -n ca-auditnet-management --
secret-name db-password --query value -o tsv)
psql "host=psql-auditnet-lcaruso.postgres.database.azure.com dbname=tfi_auditnet
user=auditnetadmin sslmode=require" -f scripts/datos-demo/historia.sql
unset PGPASSWORD
python3 scripts/datos-demo/cargar.py --url $SITIO --admin <usuario_admin> --correo
<caixa>@gmail.com --reportes
```

Problemas reais e como foram resolvidos

SINTOMA	CAUSA	SOLUÇÃO
`No plugin found for prefix 'quarkus'`	`mvn quarkus:dev` executado a partir da raiz do repositório	Executá-lo dentro de `backend/<servico>`.
O audit-core não inicia na primeira vez	As suas migrações referenciam tabelas do management	Subir primeiro o management-service.
O driver do MongoDB recusa o Cosmos DB	O Azure criou a conta com a API 3.6 e o driver exige 4.2	Atualizá-la no portal (*Upgrade MongoDB server version*); `04-crear-cosmos.sh` verifica isso.
`SubscriptionNotFound` ao criar um recurso	O provedor desse tipo de recurso não estava registrado na assinatura	`variables.sh` os registra antes (`registrar_proveedores`).
`ExpressEnvironmentResourceNotSupported`	O ambiente de Container Apps ficou no modo *express*, que não aceita jobs agendados	A revisão periódica é agendada com uma Logic App (`09-crear-revision.sh`).
O teste de fumaça dá `HTTP 000` e a web não tem endereço	No ambiente express do Container Apps, atualizar a imagem deixou a app web sem entrada	`03-actualizar-imagenes.sh` verifica a entrada de cada app e a habilita de novo. Manualmente: `az containerapp ingress enable -n ca-auditnet-web -g rg-auditnet --type external --target-port 8080`.
A primeira tela demora vários segundos	As apps escalaram para zero ou o PostgreSQL está parado	`07-modo-demo.sh on` antes de apresentar.
O Cloud Shell não encontra o repositório nem as variáveis	A sessão do Cloud Shell é efêmera	Clonar ou `git pull` e definir as variáveis novamente em cada sessão.

SINTOMA	CAUSA	SOLUÇÃO
`unrecognized arguments` em `az postgres flexible-server firewall-rule`	A CLI mudou a sintaxe	Usar `--server-name <servidor> --name <regra>`. No Cloud Shell não é preciso regra: ele entra pela de serviços do Azure.
Os e-mails de boas-vindas demoram a chegar	O domínio gerenciado do Azure limita a 10 envios por hora	Aguardar; a caixa de saída tenta de novo. Para mais volume, um domínio próprio verificado.
Um script do site não roda no Azure, mas roda em desenvolvimento	A política de segurança bloqueia scripts escritos dentro do HTML	Scripts em arquivos separados; Astro configurado com `assetsInlineLimit: 0`.
`wait_for_function` falha nos testes de tela	Ele avalia texto como código e a política o bloqueia	Usar `pagina.esperar(...)`.
O Playwright não encontra o navegador	A versão instalada não coincide com a do pacote	`python -m playwright install chromium`.
A conta está bloqueada ao testar um script	5 senhas erradas seguidas	Aguardar 15 minutos ou desbloquear em Usuários.
Depois de implantar, aparece a versão anterior	O navegador guardou a página	Recarregar com Ctrl+F5 (os arquivos de `/_astro/` têm hash; o HTML é sempre revalidado).
Sua sessão terminou sem ter saído	O perfil, a organização ou a senha mudaram, ou o token venceu	Entrar novamente: é o comportamento esperado.

Permissões por perfil e onde são controladas

MÓDULO · AÇÃO	ADM	ORG	ANL	AUD	RSP
Painel, estado do cliente, dispositivos, baselines, regras, histórico, achados, alertas e relatórios · consultar	✔	✔	✔	✔	✔
Dispositivos · registrar, editar e desativar	✔	✔			
Tipos de dispositivo · registrar	✔				
Baselines e regras · criar, versionar e editar	✔			✔	
Configurações · importar; auditorias · executar	✔		✔	✔	
Achados · alterar o estado de acompanhamento	✔		✔	✔	
Alertas críticos · encaminhar ou devolver	✔		✔		✔
Relatórios · gerar	✔		✔	✔	
Organizações · registrar, editar e desativar	✔				
Sedes · registrar, editar e desativar	✔	✔			
Usuários · gerenciar administradores	✔				
Usuários · gerenciar perfis de operação	✔	✔			
Plano · consultar os pedidos	✔	✔			✔
Plano · solicitar uma mudança		✔			
Planos · atribuir, aprovar pedidos e editar preços	✔				
Registros e painel de administração	✔				

ADM: administrador do sistema · ORG: administrador da organização · ANL: analista de redes · AUD: auditor técnico · RSP: responsável pela gestão de TI.

› Onde é controlado

CAMADA	ONDE	O QUE CONTROLA
API	`@RolesAllowed` em cada recurso de `api/rest`	Quais perfis podem chamar cada endpoint (403 se não puderem).
Escopo	`UsuarioAutenticado` (audit-core), `Actor` e serviços de usuários e sedes (management)	Que cada perfil, exceto o administrador do sistema, opere apenas sobre a sua organização.
Plano	`ControlPlan` e `PlanService`	Limites de dispositivos e usuários e o modo consulta do Piloto vencido.

CAMADA	ONDE	O QUE CONTROLA
Documentação	`MatrizPermisos` e `RolServiceTest`	A tabela mostrada na tela de Usuários; deve ser atualizada se uma permissão mudar.
Console	`Sidebar.astro` (perfis de cada item) e atributos `data-roles` / `data-admin-only`	Quais menus e botões são mostrados. É só conveniência: a segurança é aplicada pela API.
Testes	`test_perfiles.py`, `test_pantallas.py` e os testes de cada serviço	Que cada perfil veja e possa fazer o que lhe cabe.